

2 03/09/2020

2.1 Traccia da 12 punti

2.1.1 2pt

La funzione `mySum` riceve in input due matrici quadrate di interi A (di dimensione $nA \times nA$) e B (di dimensione $nB \times nB$). Considerando per ciascuna matrice le righe e le colonne, sono possibili le seguenti coppie dove il primo elemento della coppia appartiene ad A e il secondo a B : $(\text{riga}_i A, \text{riga}_j B)$, $(\text{riga}_i A, \text{col}_t B)$, $(\text{col}_k A, \text{riga}_j B)$, $(\text{col}_k A, \text{col}_t B)$ dove $0 \leq i < nA, 0 \leq k < nA, 0 \leq j < nB, 0 \leq t < nB$. Si scriva un programma che identifica la coppia in cui la somma di tutti i valori del primo e del secondo elemento (siano essi righe o colonne) sia la più vicina possibile a zero. La scelta per ogni matrice sia espressa come stringa, nella forma $R\langle\text{indice}\rangle$ ($C\langle\text{indice}\rangle$) per indicare un indice di riga (colonna). Gli indici partono da zero.

Esempio

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 3 & 2 & -3 \\ 11 & 12 & -6 \end{pmatrix} \quad B = \begin{pmatrix} 2 & 0 \\ 7 & 1 \end{pmatrix}$$

La coppia identificata dall'algoritmo è $(\text{col}_2 A, \text{riga}_1 B)$ per cui vale $(3 - 3 - 6) + (7 + 1) = 2$. L'output è quindi $C2 \ R1$. Si completi adeguatamente il codice della funzione, dato il prototipo:

```
coppia mySum(int **A, int nA, int **B, int nB)
```

e la definizione:

```
typedef struct coppia_ {char *scelta_A; char *scelta_B;} coppia;
```

Si riporti esplicitamente l'invocazione della funzione `mySum` fatta dal main chiamante.

2.1.2 4pt

Si definisca una struttura dati adatta a rappresentare una lista linkata circolare. In una lista circolare il puntatore al successore dell'elemento in coda punta all'elemento di testa della lista stessa. I nodi della lista siano caratterizzati da un valore intero e da un contatore di occorrenze. Si definisca la lista come ADT di I classe e il tipo nodo come quasi ADT. Indicare esplicitamente in quale modulo/file appare la definizione dei tipi proposti. **Non è ammesso l'uso di funzioni di libreria.** Si implementi la seguente funzione di inserimento nella lista definita in precedenza: se l'elemento è

già presente, si incrementa il contatore. Se non è presente va inserito nella posizione indicata dal secondo parametro. La testa ha posizione 0 per convenzione. **Non è ammesso l'uso di funzioni di libreria.**

```
void LISTinsert(list_t L, int posizione, ...)
```

2.1.3 6pt

Un locale ha a disposizione un insieme di nS sale. La capienza di ogni sala è registrata in un vettore S . Il locale riceve, per un certo giorno, prenotazioni da nP gruppi. La cardinalità di ciascun gruppo è registrata in un vettore P .

Esempio

4 sale di capacità, rispettivamente 4, 9, 11 e 5 persone:

$$nS = 4$$
$$S = \{4, 9, 11, 5\}$$

Prenotazione da parte di 9 gruppi con le rispettive cardinalità:

$$nP = 9$$
$$P = \{2, 2, 3, 2, 6, 2, 4, 4, 3\}$$

Si scriva un algoritmo ricorsivo in grado di determinare, se esiste, un'assegnazione delle prenotazioni ricevute nelle sale del locale, tale per cui il numero di sale occupate sia minimo. Si assuma non sia possibile spezzare una prenotazione su due/più sale. Il prototipo della funzione invocata sia nella forma:

```
void solve(int *S, int nS, int *P, int nP, ...);
```

Tale funzione può rappresentare un wrapper per l'effettiva funzione ricorsiva. Giustificare la scelta del modello combinatorio adottato. Giustificare la scelta del/dei criteri di pruning adottati, o il motivo della loro assenza.

2.2 Traccia da 18 punti

2.2.1 Breve introduzione e definizioni

Una matrice rettangolare di dimensione $R \times V$ rappresenta una griglia di gioco, le cui celle o sono vuote o contengono una cifra tra 1 e 9. La griglia di gioco è riportata su un file testuale (`griglia.txt`) con il seguente formato:

- sulla prima riga appare una coppia di interi $\langle R \rangle$ e $\langle C \rangle$ a rappresentare le dimensioni della griglia
- seguono R righe di C caratteri (tra 1 e 9 per le celle piene oppure 0 per quelle vuote).

Esempio

```
3 3
1 5 0
0 0 0
0 0 0
```

Lo scopo del gioco è completare la griglia riempiendo le celle vuote con valori tra 1 e 9 tali da creare regioni contigue di dimensione pari alle cifre in esse contenute. Si ricordi che:

- si considerano adiacenti due celle che condividono un lato verticale o orizzontale. Non sono adiacenti le celle in diagonale. La nozione di adiacenza permette quindi di considerare la mappa come un grafo (implicito) nel quale le celle libere rappresentano i vertici, mentre le adiacenze (al massimo 4 per ogni vertice) rappresentano gli archi.
- insiemi di celle adiacenti a 2 a 2 e contenenti lo stesso valore definiscono una regione contigua
- la dimensione di una regione contigua è il numero di celle che la compongono.

Alcuni dei completamenti validi per la griglia parziale precedente sono, ad esempio:

$$\begin{pmatrix} 1 & 5 & 1 \\ 5 & 5 & 5 \\ 5 & 2 & 2 \end{pmatrix} \begin{pmatrix} 1 & 5 & 3 \\ 5 & 5 & 3 \\ 5 & 5 & 3 \end{pmatrix} \begin{pmatrix} 1 & 5 & 1 \\ 5 & 5 & 5 \\ 1 & 5 & 1 \end{pmatrix} \begin{pmatrix} 1 & 5 & 5 \\ 3 & 5 & 5 \\ 3 & 3 & 5 \end{pmatrix}$$

Si scriva un programma in C che:

- definisca una opportuna struttura dati in grado di rappresentare lo schema di gioco, utilizzabile
- per risolvere i problemi che seguono
- verifichi che una griglia già completata contenuta in un file testuale rispetti le regole del gioco dato uno schema parziale, trovi (se esiste) una soluzione ottima per il problema.

2.2.2 Struttura dati

Si definiscano una struttura dati adatta a rappresentare lo schema di gioco e si definisca la funzione di acquisizione della griglia. Il prototipo della funzione sia:

```
int** leggiMappa(FILE *fin, int *R, int *C);
```

2.2.3 Problema di verifica

Si scriva una funzione che, data una griglia di gioco già completata, verifichi se questa rispetta la regola di completamento di cui sopra. Il prototipo della funzione sia:

```
int verifica(char *fn, int **mappa);
```

2.2.4 Problema di ricerca

Si scriva una funzione ricorsiva in grado di completare uno schema parziale rispettando la regola di completamento di cui sopra, ammesso che una soluzione esista. A fronte di più soluzioni possibili, l'algoritmo prediliga soluzioni che minimizzino il numero di valori distinti usati. Il prototipo della funzione sia:

```
void solve(int **m, int R, int C);
```
